Composable Scheduler Activations for Haskell

KC Sivaramakrishnan
Purdue University
chandras@cs.purdue.edu

Tim Harris[1]
Oracle Labs
timothy.l.harris@oracle.com

Simon Marlow[1]
Facebook UK Ltd.
smarlow@fb.com

Simon Peyton Jones
Microsoft Research, Cambridge
simonpj@microsoft.com

Abstract

The runtime for a modern, concurrent, garbage collected language like Java or Haskell is like an operating system: sophisticated, complex, performant, but alas very hard to change. If more of the runtime system were in the high level language, it would be far more modular and malleable. In this paper, we describe a novel concurrency substrate design for the Glasgow Haskell Compiler (GHC) that allows multicore schedulers for concurrent and parallel Haskell programs to be safely and modularly described as libraries in Haskell. The approach relies on abstracting the interface to the user-implemented schedulers through scheduler activations, together with the use of Software Transactional Memory (STM) to promote safety in a multicore context.

1. Introduction

High performance, multicore-capable runtime systems (RTS) for garbage-collected languages have been in widespread use for many years. Examples include virtual machines for popular object-oriented languages such as Oracle's Java HotSpot VM [12], IBM's Java VM [13], Microsoft's Common Language Runtime (CLR) [19], as well as functional language runtimes such as Manticore [22], MultiMLton [27] and the Glasgow Haskell Compiler (GHC) [8].

These runtime systems tend to be complex monolithic pieces of software, written not in the high-level source language (Java, Haskell, etc), but in an unsafe, systems programming language (usually C or C++). They are highly concurrent, with extensive use of locks, condition variables, timers, asynchronous I/O, thread pools, and other arcana. As a result, they are extremely difficult to modify, even for their own authors. Moreover, such modifications typically require a rebuild of the runtime, so it is not an easy matter to make changes on a program-by-program basis, let alone within a single program.

[1] This work was done at Microsoft Research, Cambridge.

[Copyright notice will appear here once 'preprint' option is removed.]

This lack of malleability is particularly unfortunate for the *thread scheduler*, which governs how the computational resources of the multi-core are deployed to run zillions of lightweight high-level language threads. A broad range of strategies are possible, including ones using priorities, hierarchical scheduling, gang scheduling, and work stealing. Different strategies might suit different multi-cores, or different application programs or parts thereof. *The goal of this paper is, therefore, to allow programmers to write a User Level Scheduler (ULS), as a library written the high level language itself.* Not only does this make the scheduler more modular and changeable, but it can readily be varied between programs, or even within a single program.

The difficulty is that the scheduler interacts intimately with other aspects of the runtime such as transactional memory or blocking I/O. Our main contribution is the design of an interface that allows expressive user-level schedulers to interact cleanly with these low-level communication and synchronisation primitives:

- We present a new concurrency substrate design for Haskell that allows application programmers to write schedulers for Concurrent Haskell programs in Haskell (Section 3). These schedulers can then be plugged-in as ordinary user libraries in the target program.
- By abstracting the interface to the ULS through *scheduler activations*, our concurrency substrate seamlessly integrates with the existing RTS concurrency support such as `MVars`, asynchronous exceptions [16], safe foreign function interface [17], software transactional memory [10], resumable black-holes [20], etc. The RTS makes *upcalls* to the activations whenever it needs to interact with the ULS. This design absolves the scheduler writer from having to reason about the interaction between the ULS and the RTS, and thus lowering the bar for writing new schedulers.
- Concurrency primitives and their interaction with the RTS are particularly tricky to specify and reason about. An unusual feature of this paper is that we precisely formalise not only the concurrency substrate primitives (Section 5), but also their interaction with the RTS concurrency primitives (Section 6).
- We present an implementation of our concurrency substrate in GHC. Experimental evaluation indicate that the

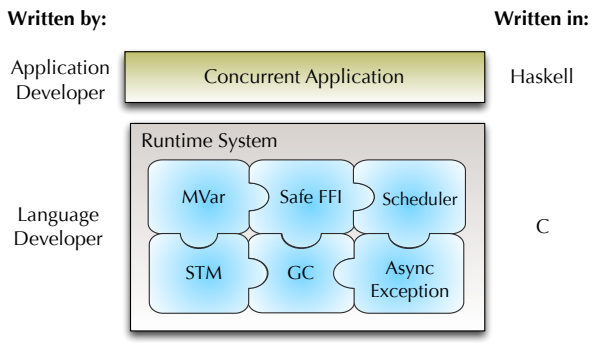

Figure 1. The anatomy of the Glasgow Haskell Compiler runtime system

performance of ULS's is comparable to the highly optimised default scheduler of GHC (Section 7).

2. Background

To understand the design of the new concurrency substrate for Haskell, we must first give some background on the existing RTS support for concurrency in our target platform – the Glasgow Haskell Compiler (GHC). We then articulate the goals of our concurrency substrate.

2.1 The GHC runtime system

GHC has a sophisticated, highly tuned RTS that has a rich support for concurrency with advanced features such as software transactional memory [10], asynchronous exceptions [16], safe foreign function interface [17], and transparent scaling on multicores [9]. The Haskell programmer can use very lightweight Haskell *threads*, which are executed by a fixed number of Haskell execution contexts, or *HECs*. Each HEC is in turn animated by an operating system thread; in this paper we use the term *tasks* for these OS threads, to distinguish them from Haskell threads. The choice of which Haskell thread is executed by which HEC is made by the *scheduler*.

GHC's current scheduler is written in C, and is hard-wired into the RTS (Figure 1). It uses a single run-queue per processor, and has a single, fixed notion of work-sharing to move work from one processor to another. There is no notion of thread priority; nor is there support for advanced scheduling policies such as gang or spatial scheduling. From an application developer's perspective, the lack of flexibility hinders deployment of new programming models on top of GHC such as data-parallel computations [4, 15], and applications such as virtual machines [7] and web-servers [11] that can benefit from the ability to define custom scheduling policies.

2.2 The challenge

Because there is such a rich design space for schedulers, our goal is to allow a user-level scheduler (ULS) to be written in Haskell, giving programmers the freedom to experiment with different scheduling or work-stealing algorithms. Indeed, we would like the ability to combine *multiple* ULS's in the same program. For example, in order to utilise the best scheduling strategy, a program could dynamically switch from a priority-based scheduler to gang scheduling when switching from general purpose computation to data-parallel computation. Applications might also combine the schedulers in a *hierarchical* fashion; a scheduler receives computational resources from its parent, and divides them among its children.

This goal is not not easy to achieve. The scheduler interacts intimately with other RTS components including

- `MVars` and transactional memory [10] allow Haskell threads to communicate and synchronise; they may cause threads to block or unblock.
- The garbage collector must somehow know about the run-queue on each HEC, so that it can use it as a root for garbage collection.
- Lazy evaluation means that if a Haskell thread tries to evaluate a thunk that is already under evaluation by another thread (it is a "black hole"), the former must block until the thunk's evaluation is complete [9]. Matters are made more complicated by asynchronous exceptions, which may cause a thread to abandon evaluation of a thunk, replacing the thunk with a "resumable black hole".
- A foreign-function call may block (e.g. when doing I/O). GHC's RTS has can schedule a fresh task (OS thread) to re-animate the HEC, blocking the in-flight Haskell thread, and scheduling a new one [17].

All of these components do things like "block a thread" or "unblock a thread" that require interaction with the scheduler. One possible response, taken by Li *et al* [14] is to program these components, too, into Haskell. The difficulty is that all they are intricate and highly-optimised. Moreover, unlike scheduling, there is no call from Haskell's users for them to be user-programmable.

Instead, our goal is to tease out the scheduler implementation from rest of the RTS, establishing a clear API between the two, and leaving unchanged the existing implementation of `MVars`, STM, black holes, FFI, and so on.

Lastly, schedulers are themselves concurrent programs, and they are particularly devious ones. Using the facilities available in C, they are extremely hard to get right. Given that the ULS will be implemented in Haskell, we would like to utilise the concurrency control abstractions provided by Haskell (notably transactional memory) to simplify the task of scheduler implementation.

3. Design

In this section, we describe the design of our concurrency substrate and present the concurrency substrate API. Along

the way, we will describe how our design achieves the goals put forth in the previous section.

3.1 Scheduler activation

Our key observation is that the interaction between the scheduler and the rest of the RTS can be reduced to two fundamental operations:

1. **Block operation.** The currently running thread blocks on some event in the RTS. The execution proceeds by switching to the next available thread from the scheduler.
2. **Unblock operation.** The RTS event that a blocked thread is waiting on occurs. After this, the blocked thread is resumed by adding it to the scheduler.

For example, in Haskell, a thread might encounter an empty `MVar` while attempting to take the value from it[2]. In this case, the thread performing the `MVar` read operation should block. Eventually, the `MVar` might be filled by some other thread (analogous to lock release), in which case, the blocked thread is unblocked and resumed with the value from the `MVar`. As we will see, all of the RTS interactions (as well as the interaction with the concurrency libraries) fall into this pattern.

Notice that the RTS blocking operations enqueue and dequeue threads from the scheduler. But the scheduler is now implemented as a Haskell library. So how does the RTS find the scheduler? We could equip each HEC with a fixed scheduler, but it is much more flexible to equip each Haskell *thread* with its own scheduler. That way, different threads (or groups thereof) can have different schedulers.

But what precisely is a "scheduler"? In our design, the scheduler is represented by two function values, or scheduler *activations*[3]. Every user-level thread has a *dequeue activation* and an *enqueue activation*. The activations provide an abstract interface to the ULS to which the thread belongs to. At the very least, the dequeue activation fetches the next available thread from the ULS encapsulated in the activation, and the enqueue activation adds the given thread to the encapsulated ULS. The activations are stored at known offsets in the thread object so that the RTS may find it. The RTS makes *upcalls* to the activations to perform the enqueue and dequeue operations on a ULS.

Figure 2 illustrates the modified RTS design that supports the implementation of ULS's. The idea is to have a minimal concurrency substrate which is implemented in C and is a part of the RTS. The substrate not only allows the programmer to implement schedulers as Haskell libraries, but also enables other RTS mechanisms to interface with the user-level schedulers through upcalls to the activations.

Figure 3 illustrates the steps associated with blocking on an RTS event. Since the scheduler is implemented in user-space, each HEC in the RTS is aware of only the currently running thread, say `t`. Suppose thread `t` waits for an abstract event `e` in the RTS, which is currently disabled. Since the thread `t` cannot continue until `e` is enabled, the RTS adds `t` to the queue of threads associated with `e`, which are currently waiting for `e` to be enabled. Notice that the RTS "owns" `t` at this point. The RTS now invokes the dequeue activation associated with `t`, which returns the next runnable thread from `t`'s scheduler queue, say `t'`. This HEC now switches control to `t'` and resumes execution. The overall effect of the operation ensure that although the thread `t` is blocked, `t`'s scheduler (and the threads that belong to it) is not blocked.

Figure 4 illustrates the steps involved in unblocking from an RTS event. Eventually, the disabled event `e` can become enabled. At this point, the RTS wakes up all of the threads waiting on event `e` by invoking their enqueue activation. Suppose we want to resume the thread `t` which is blocked on `e`. The RTS invokes `t`'s enqueue activation to add `t` to

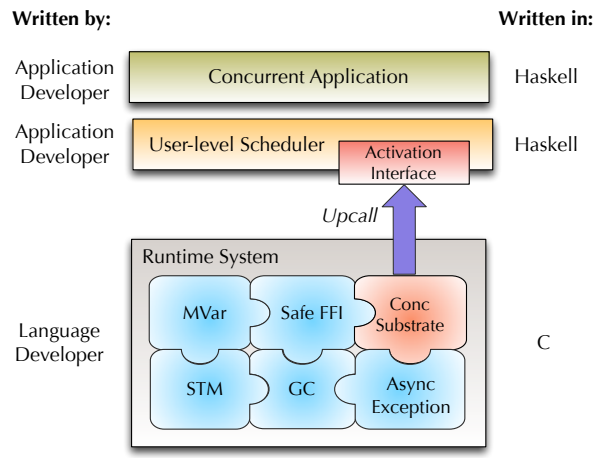

Figure 2. New GHC RTS design with Concurrency Substrate.

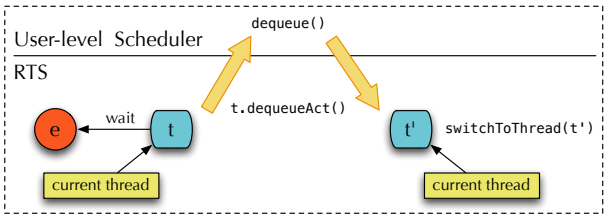

Figure 3. Blocking on an RTS event.

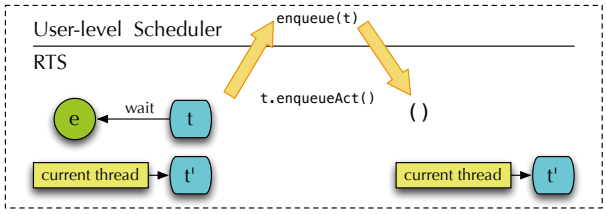

Figure 4. Unblocking from an RTS event.

[2] This operation is analogous to attempting to take a lock that is currently held by some other thread.

[3] The term "activation" comes from the operating systems literature [1]

its scheduler. Since t's scheduler is already running, t will eventually be scheduled again.

3.2 Software transactional memory

Since Haskell computations can run in parallel on different HECs, the substrate must provide a method for safely coordinating activities across multiple HECs. Similar to Li's substrate design [14], we adopt *transactional memory* (STM), as the sole multiprocessor synchronisation mechanism exposed by the substrate. Using transactional memory, rather than locks and condition variables make complex concurrent programs much more modular and less error-prone [10] – and schedulers are prime candidates, because they are prone to subtle concurrency bugs.

3.3 Concurrency substrate

Now that we have motivated our design decisions, we will present the API for the concurrency substrate. The concurrency substrate includes the primitives for instantiating and switching between language level threads, manipulating thread local state, and an abstraction for scheduler activations. The API is presented below:

```
data SCont
type DequeueAct = SCont -> STM SCont
type EnqueueAct = SCont -> STM ()

-- activation interface
dequeueAct :: DequeueAct
enqueueAct :: EnqueueAct

-- SCont manipulation
newSCont     :: IO () -> IO SCont
switch       :: (SCont -> STM SCont) -> IO ()
runOnIdleHEC :: SCont -> IO ()

-- Manipulating local state
setDequeueAct :: DequeueAct -> IO ()
setEnqueueAct :: EnqueueAct -> IO ()
getAux        :: SCont -> STM Dynamic
setAux        :: SCont -> Dynamic -> STM ()
```

3.3.1 Activation interface

Rather than directly exposing the notion of a "thread", the substrate offers *one-shot continuations* [3], which is of type SCont. An SCont is a heap-allocated object representing the current state of a Haskell computation. In the RTS, SConts are represented quite conventionally by a heap-allocated Thread Storage Object (TSO), which includes the computations stack and local state, saved registers, and program counter. Unreachable SConts are garbage collected.

The call (dequeueAct s) invokes s's dequeue activation, passing s to it like a "self" parameter. The return type of dequeueAct indicates that the computation encapsulated in the dequeueAct is transactional (under STM monad[4]), which when discharged, returns an SCont. Similarly, the call (enqueueAct s) invokes the enqueue activation transactionally, which enqueues s to its ULS.

[4] http://hackage.haskell.org/package/stm-2.1.1.0/docs/Control-Concurrent-STM.html

Since the activations are under STM monad, we have the assurance that the ULS' cannot be built with low-level unsafe components such as locks and condition variables. Such low-level operations would be under IO monad, which cannot be part of an STM transaction. Thus, our concurrency substrate statically prevents the implementation of potentially unsafe schedulers.

3.3.2 SCont management

The substrate offers primitives for creating, constructing and transferring control between SConts. The call (newSCont M) creates a new SCont that, when scheduled, executes M. By default, the newly created SCont is associated with the ULS of the invoking thread. This is done by copying the invoking SCont's activations.

An SCont is scheduled (i.e. is given control of a HEC) by the switch primitive. The call (switch M) applies M to the current continuation s. Notice that $(M\ s)$ is an STM computation. In a single atomic transaction switch performs the computation $(M\ s)$, yielding an SCont s', and switches control to s'. Thus, the computation encapsulated by s' becomes the currently running computation on this HEC.

Since our continuations are one-shot, capturing a continuation simply fetches the reference to the underlying TSO object. Hence, continuation capture involves no copying, and is cheap. Using the SCont interface, a cooperative scheduler can be built as follows:

```
yield :: IO ()
yield = switch (\s -> enqueueAct s >> dequeueAct s)
```

3.4 Parallel SCont execution

When the program begins execution, a fixed number of HECs (N) is provided to it by the environment. This signifies the maximum number of *parallel* computations in the program. Of these, one of the HEC runs the main IO computation. All other HECs are in idle state. The call runOnIdleHEC s initiates parallel execution of SCont s on an idle HEC. Once the SCont running on a HEC finishes evaluation, the HEC moves back to the idle state.

Notice that the upcall from the RTS to the dequeue activation as well as the body of the switch primitive return an SCont. This is the SCont to which the control would switch to subsequently. But what if such an SCont cannot be found? This situation can occur during multicore execution, when the number of available threads is less than the number of HECs. If a HEC does not have any work to do, it better be put to sleep.

Notice that the result of the dequeue activation and the body of the switch primitive are STM transactions. GHC today supports blocking operations under STM. When the programmer invokes retry inside a transaction, the RTS blocks the thread until another thread writes to any of the transactional variables read by the transaction; then the thread is re-awoken, and retries the transaction [10]. This

is entirely transparent to the programmer. Along the same lines, we interpret the use of `retry` within a `switch` or dequeue activation transaction as putting the whole HEC to sleep. We use the existing RTS mechanism to resume the thread when work becomes available on the scheduler.

3.5 `SCont` local state

The activations of an `SCont` can be read by `dequeueAct` and `enqueueAct` primitives. In effect, they constitute the `SCont`-local state. Local state is often convenient for other purposes, so we also provide a single dynamically-typed[5] field, the "aux-field", for arbitrary user purposes. The aux-field can be read from and written to using the primitives `getAux` and `setAux`. The API additionally allows an `SCont` to change its own scheduler through `setDequeueAct` and `setEnqueueAct` primitives.

4. Developing concurrency libraries

In this section, we will utilise the concurrency substrate to implement a multicore capable, round-robin, work-sharing scheduler and a user-level MVar implementation.

4.1 User-level scheduler

The first step in designing a scheduler is to describe the scheduler data structure. We utilise an array of runqueues, with one queue per HEC. Each runqueue is represented by a transactional variable (a `TVar`), which can hold a list of `SConts`.

```
newtype Sched = Sched (Array Int (TVar[SCont]))
```

The next step is to provide an implementation for the scheduler activations.

```
dequeueActivation:: Sched -> SCont -> STM SCont
dequeueActivation (Sched pa) _ = do
  cc <- getCurrentHEC -- get current HEC number
  l <- readTVar $ pa!cc
  case l of
    [] -> retry
    x:tl -> do
      writeTVar (pa!cc) tl
      return x

enqueueActivation:: Sched -> SCont -> STM ()
enqueueActivation (Sched pa) sc = do
  dyn <- getAux sc
  let (hec::Int, _::TVar Int) = fromJust $
      fromDynamic dyn
  l <- readTVar $ pa!hec
  writeTVar (pa!hec) $ l++[sc]
```

`dequeueActivation` either returns the `SCont` at the front of the runqueue and updates the runqueue appropriately, or puts the HEC to sleep if the queue is empty. Recall that performing `retry` within a dequeue activation puts the HEC to sleep. The HEC will automatically be woken up when work becomes available i.e. queue becomes non-empty. Although we ignore the `SCont` being blocked in this case, one could imagine manipulating the blocked `SCont`'s aux state for accounting information such as time slices consumed for fair-share scheduling. Enqueue activation (`enqueueActivation`) finds the `SCont`'s HEC number by querying its stack-local state (the details of which is presented along with the next primitive). This HEC number (`hec`) is used to fetch the correct runqueue, to which the `SCont` is appended to.

The next step is to initialise the scheduler. This involves two steps: (1) allocating the scheduler (`newScheduler`) and initialising the main thread and (2) spinning up additional HECs (`newHEC`). We assume that the Haskell program wishing to utilise the ULS performs these two steps at the start of the `main` IO computation. The implementation of these primitives are given below:

```
newScheduler :: IO ()
newScheduler = do
  -- Initialise Auxiliary state
  switch $ \s -> do
    counter <- newTVar (0::Int)
    setAux s $ toDyn $ (0::Int,counter)
    return s
  -- Allocate scheduler
  nc <- getNumHECs
  sched <- (Sched . listArray (0,nc-1)) <$>
           replicateM n (newTVar [])
  -- Initialise activations
  setDequeueAct s $ dequeueActivation sched
  setEnqueueAct s $ enqueueActivation sched

newHEC :: IO ()
newHEC = do
 -- Initial task
 s <- newSCont $ switch dequeueAct
 -- Run in parallel
 runOnIdleHEC s
```

First we will focus on initialising a new ULS (`newScheduler`). For load balancing purposes, we will spawn threads in a round-robin fashion over the available HECs. For this purpose, we initialise a `TVar` counter, and store into the auxiliary state a pair (c, t) where c is the `SCont`'s home HEC and t is the counter for scheduling. Next, we allocate an empty scheduler data structure (`sched`), and register the current thread with the scheduler activations. This step binds the current thread to participate in user-level scheduling.

All other HECs act as workers (`newHEC`), scheduling the threads that become available on their runqueues. The initial task created on the HEC simply waits for work to become available on the runqueue, and switches to it. Recall that allocating a new `SCont` copies the current `SCont`'s activations to the newly created `SCont`. In this case, the `main` `SCont`'s activations, initialised in `newScheduler`, are copied to the newly allocated `SCont`. As a result, the newly allocated `SCont` shares the same ULS with the main `SCont`. Finally, we run the new `SCont` on a free HEC. Notice that scheduler data structure is not directly accessed in `newHEC`, but accessed through the activation interface.

The Haskell program only needs to prepend the following snippet to the `main` IO computation to utilise the ULS implementation.

[5] http://hackage.haskell.org/package/base-4.6.0.1/docs/Data-Dynamic.html

```
main = do
  newScheduler
  n <- getNumHECs
  replicateM_ (n-1) newHEC
  ... -- rest of the main code
```

How do we create new user-level threads in this scheduler? For this purpose, we implement a `forkIO` primitive that spawns a new user-level thread as follows:

```
forkIO :: IO () -> IO SCont
forkIO task = do
  numHECs <- getNumHECs
  -- epilogue: Switch to next thread
  newSC <- newSCont (task >> switch dequeueAct)
  -- Create and initialise new Aux state
  switch $ \s -> do
    dyn <- getAux s
    let (_::Int, t::TVar Int) = fromJust $
         fromDynamic dyn
    nextHEC <- readTVar t
    writeTVar t $ (nextHEC + 1) `mod` numHECs
    setAux newSC $ toDyn (nextHEC, t)
    return s
  -- Add new thread to scheduler
  atomically $ enqueueAct newSC
  return newSC
```

`forkIO` primitive spawns a new thread that runs concurrently with its parent thread. What should happen after such a thread has run to completion? We must request the scheduler to provide us the next thread to run. This is captured in the epilogue `e`, and is appended to the given IO computation `task`. Next, we allocate a new `SCont`, which implicitly inherits the current `SCont`'s scheduler activations. In order to spawn threads in a round-robin fashion, we create a new auxiliary state for the new `SCont` and prepare it such that when unblocked, the new `SCont` is added to the runqueue on HEC `nextHEC`. Finally, the newly created `SCont` is added to the scheduler using its enqueue activation.

The key aspect of this `forkIO` primitive is that it does not directly access the scheduler data structure, but does so only through the activation interface. As a result, aside from the auxiliary state manipulation, the rest of the code pretty much can stay the same for any user-level `forkIO` primitive. Additionally, we can implement a `yield` primitive similar to the one described in Section 3.3.2. Due to scheduler activations, the interaction with the RTS concurrency mechanisms come for free, and we are done!

4.2 Scheduler agnostic user-level MVars

Our scheduler activations abstracts the interface to the ULS's. This fact can be exploited to build scheduler agnostic implementation of user-level concurrency libraries such as MVars. The following snippet describes the structure of an MVar implementation:

```
newtype MVar a = MVar (TVar (MVPState a))
data MVPState a = Full a [(a, SCont)]
                | Empty [(IORef a, SCont)]
```

MVar is either empty with a list of pending takers, or full with a value and a list of pending putters. An implementation of `takeMVar` function is presented below:

```
takeMVar :: MVar a -> IO a
takeMVar (MVar ref) = do
  h <- atomically $ newTVar undefined
  switch $ \s -> do
    st <- readTVar ref
    case st of
      Empty ts -> do
        writeTVar ref $ Empty $ enqueue ts (h,s)
        dequeueAct s
      Full x ts -> do
        writeTVar h x
        case deque ts of
          (_, Nothing) -> do
            writeTVar ref $ Empty emptyQueue
          (ts', Just (x', s')) -> do
            writeTVar ref $ Full x' ts'
            enqueueAct s'
        return s
  atomically $ readTVar h
```

If the MVar is empty, the `SCont` enqueues itself into the queue of pending takers. If the MVar is full, `SCont` consumes the value and unblocks the next waiting putter `SCont`, if any. The implementation of `putMVar` is the dual of this implementation. Notice that the implementation only uses the activations to block and resume the `SConts` interacting through the MVar. This allows threads from different ULS's to communicate over the same MVar, and hence the implementation is scheduler agnostic.

5. Semantics

In this section, we present the formal semantics of the concurrency substrate primitives introduced in Section 3.3. We will subsequently utilise the semantics to formally describe the interaction of the ULS with the RTS in Section 6. Our semantics closely follows the implementation. The aim of this is to precisely describe the issues with respect to the interactions between the ULS and the RTS, and have the language to enunciate our solutions.

5.1 Syntax

Figure 5 shows the syntax of program states. The program state P is a soup S of HECs, and a shared heap Θ. The operator $\|$ in the HEC soup is associative and commutative. Each HEC is either idle (`Idle`) or a triple $\langle s, M, D\rangle_t$ where s is a unique identifier of the currently executing `SCont`, M is the currently executing term, D represents `SCont`-local state. Each HEC has an optional subscript t representing its current state, and the absence of the subscript represents a HEC that is running. As mentioned in Section 3.4, when the program begins execution, the HEC soup has the following configuration:

$$\text{Initial HEC Soup } S = \langle s, M, D\rangle \parallel \texttt{Idle}_1 \parallel \ldots \parallel \texttt{Idle}_{N-1}$$

where M is the main computation, and all other HECs are idle. We represent the stack local state D as a tuple with two terms and a name (M, N, r). Here, M, N, and r are the dequeue activation, enqueue activation, and a `TVar` representing the auxiliary storage of the current `SCont` on this HEC. For perspicuity, we define accessor functions as shown below.

$$x, y \in Variable \quad r, s, \in Name$$

$$
\begin{array}{lcl}
Md & ::= & \mathsf{return}\ M \mid M \gg\!= N \\
Ex & ::= & \mathsf{throw}\ M \mid \mathsf{catch}\ M\ N \mid \mathsf{catchSTM}\ M\ N \\
Stm & ::= & \mathsf{newTVar}\ M \mid \mathsf{readTVar}\ r \mid \mathsf{writeTVar}\ r\ M \\
 & \mid & \mathsf{atomically}\ M \mid \mathsf{retry} \\
Sc & ::= & \mathsf{newSCont}\ M \mid \mathsf{switch}\ M \mid \mathsf{runOnIdleHEC}\ s \\
Sls & ::= & \mathsf{getAux}\ s \mid \mathsf{setAux}\ s\ M \\
Act & ::= & \mathsf{dequeueAct}\ s \mid \mathsf{enqueueAct}\ s \\
 & \mid & \mathsf{setDequeueAct}\ M \mid \mathsf{setEnqueueAct}\ M
\end{array}
$$

Term

$$
\begin{array}{lcl}
M, N & ::= & r \mid x \mid \lambda x \text{ -> } M \mid M\ N \mid \ldots \\
 & \mid & Md \mid Ex \mid Stm \mid Sc \mid Sls \mid Act
\end{array}
$$

$$
\begin{array}{rcccl}
\text{Program state} & P & ::= & S; \Theta \\
\text{HEC soup} & S & ::= & \emptyset \mid H \parallel S \\
\text{HEC} & H & ::= & \langle s,M,D \rangle \mid \langle s,M,D \rangle_{Sleeping} \\
 & & \mid & \langle s,M,D \rangle_{Outcall} \mid \texttt{Idle} \\
\text{Heap} & \Theta & ::= & r \mapsto M \oplus s \mapsto (M, D) \\
\text{SLS Store} & D & ::= & (M, N, r) \\
\text{IO Context} & \mathbb{E} & ::= & \bullet \mid \mathbb{E} \gg\!= M \mid \mathsf{catch}\ \mathbb{E}\ M \\
\text{STM Context} & \mathbb{P} & ::= & \bullet \mid \mathbb{P} \gg\!= M
\end{array}
$$

Figure 5. Syntax of terms, states, contexts, and heaps

$$\mathsf{deq}(M, _, _) = M \quad \mathsf{enq}(_, M, _) = M \quad \mathsf{aux}(_, _, r) = r$$

Since the semantics of primitives that read and write from `SCont`-local states (the ones under the terms Sls and Act in Figure 5) is straight-forward, and do not deter the understanding of the rest of the system, for want of space, we discuss them in the Appendix.

The heap Θ is a disjoint finite map of:

- $(r \mapsto M)$, maps the identifier r of a transactional variable, or `TVar`, to its value.
- $(s \mapsto (M, D))$, maps the identifier s of an `SCont` to its current state.

In a program state $(S; \Theta)$, an `SCont` with identifier s appears *either* as the running `SCont` in a HEC $\langle s, M, D \rangle_t \in S$, *or* as a binding $s \mapsto (M, D)$ in the heap Θ, but never in both. The distinction has direct operational significance: an `SCont` running in a HEC has part of its state loaded into machine registers, whereas one in the heap is entirely passive. In both cases, however, the term M has type `IO()`, modelling the fact that concurrent Haskell threads can perform I/O.

The number of HECs remains constant, and HEC runs one, and only one `SCont`. The business of multiplexing multiple `SConts` onto a single HEC is what the scheduler is for, and is organised by Haskell code using the primitives described in this section.

5.2 Basic transitions

Some basic transitions are presented in Figure 6. The program makes a transition from one state to another through

$$\boxed{\text{Top-level transitions} \quad S; \Theta \xRightarrow{a} S'; \Theta'}$$

$$\frac{H; \Theta \xRightarrow{a} H'; \Theta'}{H \parallel S; \Theta \xRightarrow{a} H' \parallel S; \Theta'} \ (\textsc{OneHEC})$$

$$\boxed{\text{HEC transitions} \quad H; \Theta \Longrightarrow H'; \Theta'}$$

$$\frac{M \to N}{\langle s, \mathbb{E}[M], D \rangle; \Theta \ \Longrightarrow\ \langle s, \mathbb{E}[N], D \rangle; \Theta'} \ (\textsc{PureStep})$$

$$\boxed{\text{Purely functional transitions} \quad M \to N}$$

$$
\begin{array}{rcll}
\mathsf{return}\ N \gg\!= M & \to & M\ N & (\textsc{Bind}) \\
\mathsf{throw}\ N \gg\!= M & \to & \mathsf{throw}\ N & (\textsc{Throw}) \\
\mathsf{retry} \gg\!= M & \to & \mathsf{retry} & (\textsc{Retry}) \\
\mathsf{catch}\ (\mathsf{return}\ M)\ N & \to & \mathsf{return}\ M & (\textsc{IOCatch}) \\
\mathsf{catch}\ (\mathsf{throw}\ M)\ N & \to & N\ M & (\textsc{IOCatchExn})
\end{array}
$$

Plus the usual rules for call-by-need λ-calculus, in small-step fashion

Figure 6. Operational semantics for basic transitions

the top-level program small-step transition relation: $S; \Theta \xRightarrow{a} S'; \Theta'$. This says that the program makes a transition from $S; \Theta$ to $S'; \Theta'$, possibly interacting with the underlying RTS through action a. We return to these RTS interactions in Section 6, and we omit a altogether if there is no interaction.

Rule OneHEC says that if one HEC H can take a step with the single-HEC transition relation, then the whole machine can take a step. As usual, we assume that the soup S is permuted to bring a runnable HEC to the left-hand end of the soup, so that OneHEC can fire. Similarly, Rule PureStep enables one of the HECs to perform a purely functional transition under the evaluation context $\mathbb{E}$ (defined in Figure 5). There is no action a on the arrow because this step does not interact with the RTS. Notice that PureStep transition is only possible if the HEC is in running state (with no subscript). The purely functional transitions $M \to N$ include β-reduction, arithmetic expressions, case expressions, monadic operations return, bind, throw, catch, and so on according to their standard definitions. Bind operation on the transactional memory primitive retry simply reduces to retry (Figure 6). These primitives represent blocking actions under transactional memory and will be dealt with in Section 6.2.

5.3 Transactional memory

Since the concurrency substrate primitives utilise STM as the sole synchronisation mechanism, we will present the formal semantics of basic STM operations in this section. We will build upon the basic STM formalism to formally describe the behaviour of concurrency substrate primitives in the following sections.

Figure 7 presents the semantics of non-blocking STM operations. The semantics of blocking operations is de-

$$\boxed{\text{HEC transitions} \quad H; \Theta \implies H'; \Theta'}$$

$$\frac{s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{return}\ N; \Theta'}{\langle s, \mathbb{E}[\mathsf{atomically}\ M], D\rangle;\ \Theta \implies \langle s, \mathbb{E}[\mathsf{return}\ N], D\rangle;\ \Theta'} \quad (\text{TATOMIC})$$

$$\frac{s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{throw}\ N; \Theta'}{\langle s, \mathbb{E}[\mathsf{atomically}\ M], D\rangle;\ \Theta \implies \langle s, \mathbb{E}[\mathsf{throw}\ N], D\rangle;\ \Theta \cup (\Theta' \setminus \Theta)} \quad (\text{TTHROW})$$

$$\boxed{\text{STM transitions} \quad s, M, D; \Theta; \twoheadrightarrow M'; \Theta'}$$

$$\frac{M \to N}{s; \mathbb{P}[M]; D; \Theta \twoheadrightarrow \mathbb{P}[N]; \Theta} \quad (\text{TPURESTEP})$$

$$\frac{s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{return}\ M'; \Theta'}{s; \mathbb{P}[\mathsf{catchSTM}\ M\ N]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{return}\ M']; \Theta'} \quad (\text{TCATCH})$$

$$\frac{s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{throw}\ M'; \Theta'}{s; \mathbb{P}[\mathsf{catchSTM}\ M\ N]; D; \Theta \twoheadrightarrow \mathbb{P}[N\ M']; \Theta \cup (\Theta' \setminus \Theta)} \quad (\text{TCEXN})$$

$$\frac{s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{retry}; \Theta'}{s; \mathbb{P}[\mathsf{catchSTM}\ M\ N]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{retry}]; \Theta'} \quad (\text{TCRETRY})$$

$$\frac{r\ \ \mathsf{fresh}}{s; \mathbb{P}[\mathsf{newTVar}\ M]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{return}\ r]; \Theta[r \mapsto M]} \quad (\text{TNEW})$$

$$s; \mathbb{P}[\mathsf{readTVar}\ r]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{return}\ \Theta(r)]; \Theta \quad (\text{TREAD})$$

$$s; \mathbb{P}[\mathsf{writeTVar}\ r\ M]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{return}()]; \Theta[r \mapsto M] \quad (\text{TWRITE})$$

Figure 7. Operational semantics for software transactional memory

ferred until Section 6.2. A STM transition is of the form $s; M; D; \Theta \twoheadrightarrow M'; \Theta'$, where M is the current monadic term under evaluation, and the heap Θ binds transactional variables `TVar` locations r to their current values. The current `SCont` s and its local state D are read-only, and are not used at all in this section, but will be needed when manipulating `SCont`-local state. The reduction produces a new term M' and a new heap Θ'. Rule TPURESTEP is similar to PURESTEP rule in Figure 6. STM allows creating (TNEW), reading (TREAD), and writing (TWRITE) to transactional variables.

The most important rule is TATOMIC which combines multiple *STM* transitions into a single *program* transition. Thus, other HECs are not allowed to witness the intermediate effects of the transaction. The semantics of exception handling under STM is interesting (rules TCEXN and TTHROW). Since an exception can carry a `TVar` allocated in the aborted transaction, the effects of the current transaction are undone except for the newly allocated `TVars`. Otherwise, we would have dangling pointer corresponding to such `TVars`. Rule TCRETRY simply propagates the request to retry the transaction through the context. The act of blocking, wake up and undoing the effects of the transaction are handled in Section 6.2.

5.4 `SCont` semantics

The semantics of `SCont` primitives are presented in Figure 8. Each `SCont` has a distinct identifier s (concretely, its heap address). An `SCont`'s state is represented by the pair (M, D) where M is the term under evaluation and D is the local state.

Rule NEWSCONT binds the given IO computation and a new `SCont`-local state pair to a new `SCont` s', and returns s'. Notice that the newly created `SCont` inherits the activations of the calling `SCont`. This implicitly associates the new `SCont` with the invoking `SCont`'s scheduler.

The rules for `switch` (SWITCHSELF, SWITCH, and SWITCHEXN) begin by atomically evaluating the body of switch M applied to the current `SCont` s. If the resultant `SCont` is the same as the current one (SWITCHSELF), then we simply commit the transaction and there is nothing more to be done. If the resultant `SCont` s' is different from the current `SCont` s (SWITCH), we transfer control to the new `SCont` s' by making it the running `SCont` and saving the state of the original `SCont` s in the heap. If the switch primitive happens to throw an exception, the updates by the transaction are discarded (SWITCHEXN).

The alert reader will notice that the rules for `switch` duplicate much of the paraphernalia of an atomic transaction (Figure 7), but that is unavoidable because the `switch` to a new continuation must form part of the same transaction as the argument computation.

6. Interaction with the RTS

The key aspect of our design is composability of ULS's with the existing RTS concurrency mechanisms (Section 3.1). In this section, we will describe in detail the interaction of RTS concurrency mechanisms and the ULS's. The formalisation brings out the tricky cases associated with the interaction between the ULS and the RTS.

6.1 Timer interrupts

In GHC, concurrent threads are preemptively scheduled. The RTS maintains a timer that ticks, by default, every 20ms. On a tick, the current `SCont` needs to be de-scheduled and a new `SCont` from the scheduler needs to be scheduled. The semantics of handling timer interrupts is shown in Figure 9.

The *Tick* label on the transition arrow indicates an interaction with the RTS; we call such a label an *RTS-interaction*. In this case the RTS-interaction *Tick* indicates that the RTS wants to signal a timer tick[6]. The transition here injects `yield` into the instruction stream of the `SCont` running on

[6] Technically we should ensure that every HEC receives a tick, and of course our implementation does just that, but we elide that here.

$$\boxed{\text{HEC transitions} \quad H;\Theta \implies H';\Theta'}$$

(NEWSCONT)

$$\frac{s' \text{ fresh} \quad r \text{ fresh} \quad D' = (\mathsf{deq}(D), \mathsf{enq}(D), r)}{\langle s, \mathbb{E}[\mathsf{newSCont}\ M], D\rangle;\Theta \implies \langle s, \mathbb{E}[\mathsf{return}\ s'], D\rangle;\Theta[s' \mapsto (M, D')][r \mapsto \mathsf{toDyn}\ ()]}$$

(SWITCHSELF)

$$\frac{s; M\ s; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{return}\ s; \Theta'}{\langle s, \mathbb{E}[\mathsf{switch}\ M], D\rangle;\Theta \implies \langle s, \mathbb{E}[\mathsf{return}\ ()], D\rangle;\Theta'}$$

(SWITCH)

$$\frac{s; M\ s; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{return}\ s'; \Theta'[s' \mapsto (M', D')]}{\langle s, \mathbb{E}[\mathsf{switch}\ M], D\rangle;\Theta \implies \langle s', M', D'\rangle;\Theta'[s \mapsto (\mathbb{E}[\mathsf{return}\ ()], D)]}$$

(SWITCHEXN)

$$\frac{s; M\ s; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{throw}\ N; \Theta'}{\langle s, \mathbb{E}[\mathsf{switch}\ M], D\rangle;\Theta \implies \langle s, \mathbb{E}[\mathsf{throw}\ N], D\rangle;\Theta \cup (\Theta' \setminus \Theta)}$$

(RUNONIDLEHEC)

$$\mathtt{Idle} \parallel \langle s, \mathbb{E}[\mathsf{runOnIdleHEC}\ s'], D\rangle;\Theta[s' \mapsto (M', D')] \implies \langle s', M', D'\rangle \parallel \langle s, \mathbb{E}[\mathsf{return}\ ()], D\rangle;\Theta$$

$$\langle s, \mathsf{return}\ (), D\rangle;\Theta \implies \mathtt{Idle};\Theta \quad \text{(DONEUNIT)}$$

$$\langle s, \mathsf{throw}\ N, D\rangle;\Theta \implies \mathtt{Idle};\Theta \quad \text{(DONEEXN)}$$

Figure 8. Operational semantics for `SCont` manipulation

$$\boxed{\text{HEC transitions} \quad H;\Theta \overset{a}{\implies} H';\Theta'}$$

$$\frac{\mathsf{yield} = \mathsf{switch}\ (\lambda s.\ \mathsf{enq}(D)\ s \gg \mathsf{deq}(D)\ s)}{\langle s, M, D\rangle;\Theta \overset{\mathit{Tick}}{\implies} \langle s, \mathsf{yield} \gg M, D\rangle;\Theta} \text{ (TICK)}$$

Figure 9. Handling timer interrupts

this HEC, at a GC safe point, where `yield` behaves just like the definition in Section 3.3.2.

6.2 STM blocking operations

As mentioned before (Section 3.4), STM supports blocking operations through the `retry` primitive. Figure 10 gives the semantics for STM `retry` operation.

6.2.1 Blocking the `SCont`

Rule TRETRYATOMIC is similar to TTHROW in Figure 7. It runs the transaction body M; if the latter terminates with `retry`, it abandons the effects embodied in Θ', reverting to Θ. But, unlike TTHROW it then uses an auxiliary rule $\overset{deq}{\hookrightarrow}$, defined in Figure 11, to fetch the next `SCont` to switch to. The transition in TRETRYATOMIC is labelled with the RTS interaction *STMBlock s*, indicating that the RTS assumes responsibility for s after the reduction.

The rules presented in Figure 11 are the key rules in abstracting the interface between the ULS and the RTS, and describe the invocation of upcalls. In the sequel, we will often refer to these rules in describing the semantics of the RTS interactions. Rule UPDEQUEUE in Figure 11 stashes s (the `SCont` to be blocked) in the heap Θ, instantiates an ephemeral `SCont` that fetches the `dequeue` activation b from s's local state D, and switches to the `SCont` returned by the

$$\boxed{\text{HEC transitions} \quad H;\Theta \overset{a}{\implies} H';\Theta'}$$

(TRETRYATOMIC)

$$\frac{\begin{array}{c} s; M; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{retry}; \Theta' \\ \langle s, \mathbb{E}[\mathsf{atomically}\ M], D\rangle;\Theta \overset{deq}{\hookrightarrow} H';\Theta'' \end{array}}{\langle s, \mathbb{E}[\mathsf{atomically}\ M], D\rangle;\ \Theta \overset{\mathit{STMBlock\ s}}{\Longrightarrow} H';\ \Theta''}$$

(TRESUMERETRY)

$$\frac{H;\Theta \overset{enq\ s}{\hookrightarrow} H';\Theta'}{H;\Theta \overset{\mathit{RetrySTM\ s}}{\Longrightarrow} H';\Theta'}$$

(TRETRYSWITCH)

$$\frac{s; M\ s; D; \Theta \overset{*}{\twoheadrightarrow} \mathsf{retry}; \Theta'}{\langle s, \mathbb{E}[\mathsf{switch}\ M], D\rangle;\Theta \overset{\mathit{STMBlock\ s}}{\Longrightarrow} \langle s, \mathbb{E}[\mathsf{switch}\ M], D\rangle_{\mathit{Sleeping}};\Theta}$$

(TWAKEUP)

$$\langle s, \mathbb{E}[M], D\rangle_{\mathit{Sleeping}};\Theta \overset{\mathit{RetrySTM\ s}}{\Longrightarrow} \langle s, \mathbb{E}[M], D\rangle;\Theta$$

Figure 10. STM Retry

`dequeue` activation. s' is made the running `SCont` on this HEC.

It is necessary that the dequeue upcall be performed on a new `SCont` s', and not on the `SCont` s being blocked. At the point of invocation of the dequeue upcall, the RTS believes that the blocked `SCont` s is completely owned by the RTS, not running, and available to be resumed. Invoking the dequeue upcall on the blocked `SCont` s can lead to a race on s between multiple HECs if s happens to be unblocked and enqueued to the scheduler before the `switch` transaction is completed.

$$\boxed{\text{Dequeue upcall instantiation} \quad H;\Theta \overset{deq}{\hookrightarrow} H';\Theta'}$$

(UPDEQUEUE)

$$\frac{\begin{array}{c} s' \text{ fresh} \quad r \text{ fresh} \quad D' = (\mathsf{deq}(D), \mathsf{enq}(D), r) \\ M' = \mathsf{switch}\ (\lambda x.\, \mathsf{deq}(D)\ s) \\ \Theta' = \Theta[s \mapsto (M, D)][r \mapsto \mathsf{toDyn}\ ()] \end{array}}{\langle s, M, D\rangle;\Theta \overset{deq}{\hookrightarrow} \langle s', M', D'\rangle;\Theta'}$$

$$\boxed{\text{Enqueue upcall instantiation} \quad H;\Theta \overset{enq\ s}{\hookrightarrow} H';\Theta'}$$

(UPENQUEUEIDLE)

$$\frac{\begin{array}{c} s' \text{ fresh} \quad r \text{ fresh} \quad D' = (\mathsf{deq}(D), \mathsf{enq}(D), r) \\ M' = \mathsf{atomically}\ (\mathsf{enq}(D)\ s) \\ \Theta' = \Theta[s \mapsto (M, D)][r \mapsto \mathsf{toDyn}\ ()] \end{array}}{\texttt{Idle};\Theta[s \mapsto (M, D)] \overset{enq\ s}{\hookrightarrow} \langle s', M', D'\rangle;\Theta'}$$

(UPENQUEUERUNNING)

$$\frac{M'' = \mathsf{atomically}\ (\mathsf{enq}(D)\ s) \gg M'}{\langle s', M', D'\rangle;\Theta[s \mapsto (M, D)] \overset{enq\ s}{\hookrightarrow} \langle s', M'', D'\rangle;\Theta[s \mapsto (M, D)]}$$

Figure 11. Instantiating upcalls

6.2.2 Resuming the `SCont`

Some time later, the RTS will see that some thread has written to one of the `TVars` read by s's transaction, so it will signal an *RetrySTM s* interaction (rule TRESUMERETRY). Again, we use an auxiliary transition $\overset{enq\ s}{\hookrightarrow}$ to enqueue the `SCont` to its scheduler (Figure 11).

Unlike $\overset{deq}{\hookrightarrow}$ transition, unblocking an `SCont` has nothing to do with the computation currently running on any HEC. If we find an idle HEC (rule UPENQUEUEIDLE), we instantiate a new ephemeral `SCont` s' to enqueue the `SCont` s. The actual unblock operation is achieved by fetching `SCont` s's `enqueue` activation, applying it to s and atomically performing the resultant STM computation. If we do not find any idle HECs (rule UPENQUEUERUNNING), we pick one of the running HECs, prepare it such that it first unblocks the `SCont` s before resuming the original computation.

6.2.3 HEC sleep and wakeup

Recall that invoking `retry` within a `switch` transaction or `dequeue` activation puts the HEC to sleep (Section 3.4). Also, notice that the `dequeue` activation is always invoked by the RTS from a `switch` transaction (Rule UPDEQUEUE). This motivates rule TRETRYSWITCH: if a `switch` transaction blocks, we put the whole HEC to sleep. Then, dual to TRESUMERETRY, rule TWAKEUP wakes up the HEC when the RTS sees that the transaction may now be able to make progress.

$$\boxed{\text{HEC transitions} \quad H;\Theta \overset{a}{\Longrightarrow} H';\Theta'}$$

$$\frac{}{\begin{array}{c}\langle s, \mathbb{E}[\mathsf{outcall}\ r], D\rangle;\Theta \overset{OC\ s}{\Longrightarrow} \\ \langle s, \mathbb{E}[\mathsf{outcall}\ r], D\rangle_{Outcall};\Theta\end{array}} \quad \text{(OCBLOCK)}$$

$$\frac{}{\begin{array}{c}\langle s, \mathbb{E}[\mathsf{outcall}\ r], D\rangle_{Outcall};\Theta \overset{OCRet\ s\ M}{\Longrightarrow} \\ \langle s, \mathbb{E}[M], D\rangle;\Theta\end{array}} \quad \text{(OCRETFAST)}$$

$$\frac{\langle s, M, D\rangle;\Theta \overset{deq}{\hookrightarrow} H';\Theta'}{\langle s, M, D\rangle_{Outcall};\Theta \overset{OCSteal\ s}{\Longrightarrow} H';\Theta'} \quad \text{(OCSTEAL)}$$

$$\frac{H;\Theta[s \mapsto (\mathbb{E}[M], D)] \overset{enq\ s}{\hookrightarrow} H';\Theta'}{H;\Theta[s \mapsto (\mathbb{E}[\mathsf{outcall}\ r], D)] \overset{OCRet\ s\ M}{\Longrightarrow} H';\Theta'} \quad \text{(OCRETSLOW)}$$

Figure 12. Safe foreign call transitions

6.2.4 Implementation of upcalls

Notice that the rules UPDEQUEUE and UPENQUEUEIDLE in Figure 11 instantiate a fresh `SCont`. The freshly instantiated `SCont` performs just a single transaction; `switch` in UPDEQUEUE and `atomically` in UPENQUEUEIDLE, after which it is garbage-collected. Since instantiating a fresh `SCont` for every upcall is unwise, the RTS maintains a dynamic pool of dedicated *upcall SConts* for performing the upcalls. It is worth mentioning that we need an "upcall `SCont` pool" rather than a single "upcall `SCont`" since the upcall transactions can themselves get blocked synchronously on STM `retry` as well as asynchronously due to optimizations for lazy evaluation (Section 6.5).

6.3 Safe foreign function calls

Foreign calls in GHC are highly efficient but intricately interact with the scheduler [17]. Much of it owes to the the RTS's task model. Each HEC is animated by one of a *pool* of tasks (OS threads); the current task may become blocked in a foreign call (e.g. a blocking I/O operation), in which case another task takes over the HEC. However, at most only one task ever has exclusive access to a HEC.

GHC's task model ensures that a HEC performing a safe-foreign call only blocks the Haskell thread (and the task) making the call but not the other threads running on the HEC's scheduler. However, it would be unwise to switch the thread (and the task) on every foreign call as most invocations are expected to return in a timely fashion. In this section, we will discuss the interaction of safe-foreign function calls and the ULS. In particular, we restrict the discussion to outcalls — calls made from Haskell to C.

Our decision to preserve the task model in the RTS allows us to delegate much of the work involved in safe foreign call to the RTS. We only need to deal with the ULS interaction, and not the creation and coordination of tasks. The semantics of foreign call handling is presented in Figure 12. Rule OCBLOCK illustrates that the HEC performing the foreign

call moves into the $Outcall$ state, where it is no longer runnable. In the fast path (rule OCRETFAST), the foreign call returns immediately with the result M, and the HEC resumes execution with the result plugged into the context.

In the slow path, the RTS may decide to pay the cost of task switching and resume the scheduler (rule OCSTEAL). The scheduler is resumed using the dequeue upcall. Once the foreign call eventually returns, the `SCont` s blocked on the foreign call can be resumed. Since we have already resumed the scheduler, the correct behaviour is to prepare the `SCont` s with the result and add it to its ULS. Rule OCRETSLOW achieves this through enqueue upcall.

6.4 Timer interrupts and transactions

What if a timer interrupt occurs during a transaction? The (TICK) rule of Section 6.1 is restricted to *HEC transitions*, and says nothing about *STM transitions*. One possibility (Plan A) is that transactions should not be interrupted, and ticks should only be delivered at the end. This is faithful to the semantics expressed by the rule, but it does mean that a rogue transaction could completely monopolise a HEC.

An alternative possibility (Plan B) is for the RTS to roll the transaction back to the beginning, and then deliver the tick using rule (TICK). That too is implementable, but this time the risk is that a slightly-too-long transaction would always be rolled back, so it would never make progress.

Our implementation behaves like Plan B, but gives better progress guarantees, while respecting the same semantics. Rather than rolling the transaction back, the RTS suspends the transaction mid-flight. None of its effects are visible to other `SConts`; they are confined to its `SCont`-local transaction log. When the `SCont` is later resumed, the transaction continues from where it left off, rather than starting from scratch. Of course, time has gone by, so when it finally tries to commit there is a higher chance of failure, but at least uncontended access will go through.

That is fine for vanilla `atomically` transactions. But what about the special transactions run by `switch`? If we are in the middle of a `switch` transaction, and suspend it to deliver a timer interrupt, rule (TICK) will initiate ... a `switch` transaction! And that transaction is likely to run the very same code that has just been interrupted. It seems much simpler to revert to Plan A: the RTS does not deliver timer interrupts during a `switch` transaction. If the scheduler has rogue code, then it will monopolise the HEC with no recourse.

6.5 Black holes

In a concurrent Haskell program, a thread `A` may attempt to evaluate a thunk `x` that is already being evaluated by another thread `B`. To avoid duplicate evaluation the RTS (in intimate cooperation with the compiler) arranges for `B` to *blackhole* the thunk when it starts to evaluate `x`. Then, when `A` attempts to evaluate `x`, it finds a black hole, so the RTS enqueues `A` to await its completion. When `B` finishes evaluating `x` it updates the black hole with its value, and makes any queued threads runnable. This mechanism, and its implementation on a multicore, is described in detail in earlier work [9].

HEC transitions $H; \Theta \xrightarrow{a} H'; \Theta'$

$$\frac{\langle s, M, D\rangle; \Theta \overset{deq}{\hookrightarrow} H'; \Theta'}{\langle s, M, D\rangle;\ \Theta \xrightarrow{\textit{BlockBH}\ s} H';\ \Theta'} \quad \text{(BLOCKBH)}$$

$$\frac{H; \Theta \overset{enq\ s}{\hookrightarrow} H'; \Theta'}{H; \Theta \xrightarrow{\textit{ResumeBH}\ s} H'; \Theta'} \quad \text{(RESUMEBH)}$$

Figure 13. Black holes

Clearly this is another place where the RTS may initiate blocking. We can describe the common case with rules similar to those of Figure 10, with rules shown in Figure 13. The RTS initiates the process with a *BlockBH* s action, taking ownership of the `SCont` s. Later, when the evaluation of the thunk is complete, the RTS initiate an action *ResumeBH* s, which returns ownership to s's scheduler.

But these rules only apply to *HEC transitions*, outside transactions. What if a black hole is encountered during an STM transaction? We addressed this same question in the context of timer interrupts, in Section 6.4, and we adopt the same solution. The RTS behaves as if the black-hole suspension and resumption occurred just before the transaction, but the implementation actually arranges to resume the transaction from where it left off.

Just as in Section 6.4, we need to take particular care with `switch` transactions. Suppose a `switch` transaction encounters a black-holed thunk under evaluation by some other `SCont B`; and suppose we try to suspend the transaction (either mid-flight or with roll-back) using rule (BLOCKBH). Then the very next thing we will do (courtesy of $\overset{deq}{\hookrightarrow}$) is a `switch` transaction; and that is very likely to encounter the very same thunk. Moreover, it is just possible that the thunk is under evaluation by an `SCont` in this very scheduler's runqueue, so the black hole is preventing us from scheduling the very `SCont` that is evaluating it. Deadlock beckons!

In the case of timer interrupts we solved the problem by switching them off in `switch` transactions, and it turns out that we can effectively do the same for thunks. Since we cannot sensibly suspend the `switch` transaction, we must find a way for it to make progress. Fortunately, GHC's RTS allows us to *steal* the thunk from the `SCont` that is evaluating it, and that suffices. The details are beyond the scope of this paper, but the key moving parts are already part of GHC's implementation of asynchronous exceptions [16, 20].

6.6 Interaction with RTS MVars

An added advantage of our scheduler activation interface is that we are able to reuse the existing MVar implementation in the RTS. Whenever an `SCont` s needs to block on or

unblock from an MVar, the RTS invokes the $\overset{deq}{\hookrightarrow}$ or $\overset{enq\ s}{\hookrightarrow}$ upcall, respectively. This significantly reduces the burden of migrating to a ULS implementation.

6.7 Asynchronous exceptions

GHC's supports *asynchronous exceptions* in which one thread can send an asynchronous interrupt to another [16]. This is a very tricky area; for example, if a thread is blocked on a user-level `MVar` (Section 4.2), and receives an exception, it should wake up and do something — even though it is linked onto an unknown queue of blocked threads. Our implementation does in fact handle asynchronous exceptions, but we are not yet happy with the details of the design, and in any case space precludes presenting them here.

6.8 On the correctness of user-level schedulers

While the concurrency substrate exposes the ability to build ULS's, the onus is on the scheduler implementation to ensure that it is sensible. The invariants such as not switching to a running thread, or a thread blocked in the RTS, are not statically enforced by the concurrency substrate, and care must be taken to preserve these invariants. Our implementation dynamically enforces such invariants through runtime assertions. We also expect that the activations do not raise an exception that escape the activation. Activations raising exceptions indicates an error in the ULS implementation, and the substrate simply reports an error to the standard error stream.

The fact that the scheduler itself is now implemented in user-space complicates error recovery and reporting when threads become unreachable. A thread suspended on an ULS may become unreachable if the scheduler data structure holding it becomes unreachable. A thread indefinitely blocked on an RTS `MVar` operation is raised with an exception and added to its ULS. This helps the corresponding thread from recovering from indefinitely blocking on an `MVar` operation.

However, the situation is worse if the ULS itself becomes unreachable; there is no scheduler to run this thread! Hence, salvaging such a thread is not possible. In this case, immediately after garbage collection, our implementation logs an error message to the standard error stream along with the unreachable `SCont` (thread) identifier.

7. Results

Our implementation is a fork of GHC,[7] and supports all of the features discussed in the paper. We have been very particular not to compromise on any of the existing features in GHC. As shown in Section 4, porting existing concurrent Haskell program to utilise a ULS only involves few additional lines of code.

[7] The development branch of LWC substrate is available at `https://github.com/ghc/ghc/tree/ghc-lwc2`

Benchmark	Baseline (1 proc)	Vanilla		LWC	
		1 HEC	Fastest (# HECs)	1 HEC	Fastest (# HECs)
`k-nucleotide`	10.60	10.62	4.82 (8)	10.61	4.83 (8)
`mandelbrot`	85.30	90.83	3.21 (48)	87.06	2.19 (48)
`spectral-norm`	125.76	125.91	2.92 (48)	125.76	2.84 (48)
`chameneos`	4.62	5.71	5.71 (1)	18.25	12.35 (2)
`primes-sieve`	32.52	36.33	36.33 (1)	223	13.7 (48)

Figure 14. Benchmark results. All times are in seconds.

In order to evaluate the performance and quantify the overheads of LWC substrate, we picked the following Haskell concurrency benchmarks from The Computer Language Benchmarks Game [24]: `k-nucleotide`, `mandelbrot`, `spectral-norm` and `chameneos`. We also implemented a concurrent prime number generator using sieve of Eratosthenes (`primes-sieve`), where the threads communicate over the MVars. For our experiments, we generated the first 10000 primes. The benchmarks offer varying degrees of parallelisation opportunity. `k-nucleotide`, `mandelbrot` and `spectral-norm` are computation intensive, while `chameneos` and `primes-sieve` are communication intensive and are specifically intended to test the overheads of thread synchronisation.

The LWC version of the benchmarks utilised the scheduler and the MVar implementation described in Section 4. For comparison, the benchmark programs were also implemented using `Control.Concurrent` on a vanilla GHC implementation. Experiments were performed on a 48-core AMD Opteron server, and the GHC version was 7.7.20130523.

The results are presented in Figure 14. For each benchmark, the baseline is the non-threaded (not compiled with `-threaded`) vanilla GHC version. The non-threaded version does not support multi-processor execution of concurrent programs, but also does not include the mechanisms necessary for (and the overheads included in) multi-processor synchronisation. Hence, the non-threaded version of a program running on 1 processor is faster than the corresponding threaded version.

For the vanilla and LWC versions (both compiled with `-threaded`), we report the running times on 1 HEC as well as the fastest running time observed with additional HECs. Additionally, we report the HEC count corresponding to the fastest configuration. All the times are reported in seconds.

In `k-nucleotide` and `spectral-norm` benchmarks, the performance of LWC version was indistinguishable from the vanilla version. The threaded versions of the benchmark programs were fastest on 8 HECs and 48 HECs on `k-nucleotide` and `spectral-norm`, respectively. In `mandelbrot` benchmark, LWC version was *faster* than the vanilla version. While the vanilla version was 29× faster than the baseline, LWC version was 38× faster. In the vanilla GHC, the RTS thread scheduler by default spawns a thread on the current HEC and only shares the thread with other HECs if they are idle. The LWC scheduler (described in Sec-

tion 4) spawns threads by default in a round-robin fashion on all HECs. This simple scheme happens to work better in `mandelbrot` since the program is embarrassingly parallel.

In `chameneos` benchmark, the LWC version was 3.9× slower than the baseline on 1 HEC and 2.6× slower on 2 HECs, and slows down with additional HECs as `chameneos` does not present much parallelisation opportunity. The vanilla `chameneos` program was fastest on 1 HEC, and was 1.24× slower than the baseline. In `primes-sieve` benchmark, while the LWC version was 6.8× slower on one HEC, the vanilla version was 1.3X slower, when compared to the baseline.

In `chameneos` and `primes-sieve`, we observed that the LWC implementation spends around half of its execution running the transactions for invoking the activations or `MVar` operations. Additionally, in these benchmarks, LWC version performs 3X-8× more allocations than the vanilla version. Most of these allocations are due to the data structure used in the ULS and the MVar queues. In the vanilla `primes-sieve` implementation, these overheads are negligible. This is an unavoidable consequence of implementing concurrency libraries in Haskell.

Luckily, these overheads are parallelisable. In `primes-sieve` benchmark, while the vanilla version was fastest on 1 HEC, LWC version scaled to 48 HECs, and was 2.37× *faster* than the baseline program. This gives us the confidence that with careful optimisations and application specific heuristics for the ULS and the MVar implementation, much of the overheads in the LWC version can be eliminated.

8. Related Work

Continuation based concurrency libraries have been well studied [23, 25] and serve as the basis of several parallel and concurrent programming language implementations [21, 22, 27]. Among these, ConcurrentML [21] implementations on SML/NJ and MLton, and MultiMLton [27] do not expose the ability to describe alternative ULS's. Fluet et al. [5] propose a scheduling framework for a strict parallel functional language on Manticore [22]. However, unlike our system, the schedulers are described in an external language of the compiler's internal representation, and not the source language.

Of the meta-circular implementations of Java, Jikes RVM [6] is perhaps the most mature. Jikes does not support user-level threads, and maps each Java thread directly on to a native thread, which are arbitrarily scheduled by the OS. This decision is partly motivated to offer better compatibility with Java Native Interface (JNI), the foreign function interface in Java. Thread-processor mapping is also transparent to the programmer. Jikes supports unsafe low-level operations to block and synchronise threads in order to implement other operations such as garbage collection. Compared to Jikes, our concurrency substrate *only* permits safe interaction with the scheduler through the STM interface. The ULS's also integrates well with GHC's safe foreign-function interface through the activation interface (Section 6.3).

While Manticore [22], and MultiMLton [27] utilise low-level compare-and-swap operation as the core synchronisation primitive, Li et al.'s concurrency substrate [14] for GHC was the first to utilise transactional memory for multiprocessor synchronisation for in the context of ULS's. Our work borrows the idea of using STM for synchronisation. Unlike Li's substrate, we retain the key components of the concurrency support in the runtime system. Not only does alleviate the burden of implementing the ULS, but enables us to safely handle the issue of blackholes that requires RTS support, and perform blocking operations under STM. In addition, Li's substrate work uses explicit wake up calls for unblocking sleeping HECs. This design has potential for bugs due to forgotten wake up messages. Our HEC blocking mechanism directly utilises STM blocking capability provided by the runtime system, and by construction eliminates the possibility of forgotten wake up messages.

Scheduler activations [1] have successfully been demonstrated to interface kernel with the user-level process scheduler [2, 26]. Similar to scheduler activations, Psyche [18] allows user-level threads to install event handlers for scheduler interrupts and implement the scheduling logic in user-space. Unlike these works, our system utilises scheduler activations in the language runtime rather than OS kernel. Moreover, our activations being STM computations allow them to be composed with other language level transactions in Haskell, enabling scheduler-agnostic concurrency library implementations.

9. Conclusions and Future Work

We have presented a concurrency substrate design for Haskell that lets programmers write schedulers for Haskell threads as ordinary libraries in Haskell. Through an activation interface, this design lets GHC's runtime system to safely interact with the user-level scheduler, and therefore tempering the complexity of implementing full-fledged schedulers. The fact that many of the RTS interactions such as timer interrupts, STM blocking operation, safe foreign function calls, etc,. can be captured through the activation interface reaffirms the idea that we are on the right track with the abstraction.

Our precise formalisation of the RTS interactions served as a very good design tool and a validation mechanism, and helped us gain insights into subtle interactions between the ULS and the RTS. Through the formalisation, we realised that the interaction of black holes and timer interrupts with a scheduler transaction is particularly tricky, and must be handled explicitly by the RTS in order to avoid livelock and deadlock.

As the next step, we plan to improve upon our current solution for handling asynchronous exceptions. A part of this solution involves making the `SCont` reference a bona

fide one-shot continuation, and not simply a reference to the underlying TSO object. As a result, concurrency substrate should be able to better handle erroneous scheduler behaviours rather than raising an error and terminating. As for the implementation, we would like to explore the effectiveness user-level gang scheduling for Data Parallel Haskell [4] workloads, and priority scheduling for Haskell based webservers [11] and virtual machines [7].

References

[1] T. E. Anderson, B. N. Bershad, E. D. Lazowska, and H. M. Levy. Scheduler activations: effective kernel support for the user-level management of parallelism. *ACM Trans. Comput. Syst.*, 10(1):53–79, Feb. 1992.

[2] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach, and A. Singhania. The multikernel: a new os architecture for scalable multicore systems. In *SOSP*, pages 29–44, 2009.

[3] C. Bruggeman, O. Waddell, and R. K. Dybvig. Representing control in the presence of one-shot continuations. In *PLDI*, pages 99–107, 1996.

[4] M. M. T. Chakravarty, R. Leshchinskiy, S. Peyton Jones, G. Keller, and S. Marlow. Data parallel haskell: a status report. In *DAMP*, pages 10–18, 2007.

[5] M. Fluet, M. Rainey, and J. Reppy. A scheduling framework for general-purpose parallel languages. In *ICFP*, pages 241–252, 2008.

[6] D. Frampton, S. M. Blackburn, P. Cheng, R. J. Garner, D. Grove, J. E. B. Moss, and S. I. Salishev. Demystifying magic: high-level low-level programming. In *VEE*, pages 81–90, 2009.

[7] Galois. Haskell Lightweight Virtual Machine (HaLVM), 2014. `http://corp.galois.com/halvm`.

[8] GHC. Glasgow Haskell Compiler, 2014. `http://www.haskell.org/ghc`.

[9] T. Harris, S. Marlow, and S. Peyton Jones. Haskell on a shared-memory multiprocessor. In *ACM Workshop on Haskell*, Tallin, Estonia, 2005.

[10] T. Harris, S. Marlow, S. Peyton-Jones, and M. Herlihy. Composable memory transactions. In *PPoPP*, pages 48–60, 2005.

[11] Haskell. Haskell Web Development, 2014. `http://www.haskell.org/haskellwiki/Web/Servers`.

[12] HotSpotVM. Java SE HotSpot at a Glance, 2014. `http://www.oracle.com/technetwork/java/javase/tech/index-jsp-137187.html`.

[13] IBM. Java Platform Standard Edition (Java SE), 2014. `http://www.ibm.com/developerworks/java/jdk/`.

[14] P. Li, S. Marlow, S. Peyton Jones, and A. Tolmach. Lightweight concurrency primitives for ghc. In *Haskell*, pages 107–118, 2007.

[15] B. Lippmeier, M. Chakravarty, G. Keller, and S. Peyton Jones. Guiding parallel array fusion with indexed types. In *Haskell*, pages 25–36, 2012.

[16] S. Marlow, S. P. Jones, A. Moran, and J. Reppy. Asynchronous exceptions in haskell. In *PLDI*, pages 274–285, 2001.

[17] S. Marlow, S. P. Jones, and W. Thaller. Extending the haskell foreign function interface with concurrency. In *Haskell*, pages 22–32, 2004.

[18] B. D. Marsh, M. L. Scott, T. J. Leblanc, and E. P. Markatos. First-class user-level threads. In *OSDI*, pages 110–121, 1991.

[19] Microsoft Corp. Common Language Runtime (CLR), 2014. `http://msdn.microsoft.com/en-us/library/8bs2ecf4(v=vs.110).aspx`.

[20] A. Reid. Putting the spine back in the spineless tagless g-machine: An implementation of resumable black-holes. In *IFL*, pages 186–199, 1999.

[21] J. Reppy. *Concurrent Programming in ML*. Cambridge University Press, 2007.

[22] J. Reppy, C. V. Russo, and Y. Xiao. Parallel concurrent ml. In *ICFP*, pages 257–268, 2009.

[23] O. Shivers. Continuations and Threads: Expressing Machine Concurrency Directly in Advanced Languages. In *Continuations Workshop*, 1997.

[24] Shootout. The Computer Language Benchmarks Game, 2014. `http://benchmarksgame.alioth.debian.org/`.

[25] M. Wand. Continuation-based multiprocessing. In *LFP*, pages 19–28, 1980.

[26] N. J. Williams. An Implementation of Scheduler Activations on the NetBSD Operating System. In *Usenix ATC*, pages 99–108, 2002.

[27] L. Ziarek, K. Sivaramakrishnan, and S. Jagannathan. Composable Asynchronous Events. In *PLDI*, pages 628–639, 2011.

Appendix

Semantics of local state manipulation

$$\boxed{\text{HEC transitions} \quad H; \Theta \implies H'; \Theta'}$$

(SETDEQUEUEACT)

$$\langle s, \mathbb{E}[\mathsf{setDequeueAct}\ M], (b,u,r)\rangle; \Theta \implies \langle s, \mathbb{E}[\mathsf{return}\,()], (M,u,r)\rangle; \Theta$$

(SETENQUEUEACT)

$$\langle s, \mathbb{E}[\mathsf{setEnqueueAct}\ M], (b,u,r)\rangle; \Theta \implies \langle s, \mathbb{E}[\mathsf{return}\,()], (b,M,r)\rangle; \Theta$$

$$\boxed{\text{STM transitions} \quad s; M; D; \Theta \twoheadrightarrow M'; \Theta'}$$

(GETAUXSELF)

$$s; \mathbb{P}[\mathsf{getAux}\ s]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{return}\ \mathsf{aux}(D)]; \Theta$$

(SETAUXSELF)

$$s; \mathbb{E}[\mathsf{setAux}\ s\ M]; D; \Theta \twoheadrightarrow \mathbb{E}[\mathsf{return}\,()]; \Theta[\mathsf{aux}(D) \mapsto M]$$

(INVOKEDEQUEUEACTSELF)

$$s; \mathbb{P}[\mathsf{dequeueAct}\ s]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{deq}(D)\ s]; \Theta$$

(INVOKEENQUEUEACTSELF)

$$s; \mathbb{P}[\mathsf{enqueueAct}\ s]; D; \Theta \twoheadrightarrow \mathbb{P}[\mathsf{enq}(D)\ s]; \Theta$$

(GETAUXOTHER)

$$s; \mathbb{P}[\mathsf{getAux}\ s']; D; \Theta[s' \mapsto (M', D')] \twoheadrightarrow \mathbb{P}[\mathsf{return}\ \mathsf{aux}(D')]; \Theta[s' \mapsto (M', D')]$$

(SETAUXOTHER)

$$s; \mathbb{E}[\mathsf{setAux}\ s'\ M]; D; \Theta[s' \mapsto (M', D')] \twoheadrightarrow \mathbb{E}[\mathsf{return}\,()]; \Theta[s' \mapsto (M', D')][\mathsf{aux}(D') \mapsto M]$$

(INVOKEDEQUEUEACTOTHER)

$$s; \mathbb{P}[\mathsf{dequeueAct}\ s']; D; \Theta[s' \mapsto (M', D')] \twoheadrightarrow \mathbb{P}[\mathsf{deq}(D')\ s']; \Theta[s' \mapsto (M', D')]$$

(INVOKEENQUEUEACTOTHER)

$$s; \mathbb{P}[\mathsf{enqueueAct}\ s']; D; \Theta[s' \mapsto (M', D')] \twoheadrightarrow \mathbb{P}[\mathsf{enq}(D')\ s']; \Theta[s' \mapsto (M', D')]$$

Figure 15. Operational semantics for manipulating activations and auxiliary state.

In our formalisation, we represent local state D as a tuple with two terms and a name (M, N, r) (Figure 5), where M, N and r are dequeue activation, enqueue activation, and a `TVar` representing auxiliary storage, respectively. For perspicuity, we define accessor functions as shown below.

$$\mathsf{deq}(M, _, _) = M \quad \mathsf{enq}(_, M, _) = M \quad \mathsf{aux}(_, _, r) = r$$

The precise semantics of activations and stack-local state manipulation is given in Figure 15. Our semantics models the auxiliary field in the `SCont`-local state as a `TVar`. It is initialised to a dynamic unit value `toDyn` () when an new `SCont` is created (rule NEWSCONT in Figure 8). The rules SETAUXSELF and SETAUXOTHER update the aux state of a `SCont` by writing to the `TVar`. There are two cases, depending on whether the `SCont` is running in the current HEC, or is passive in the heap. The aux-state is typically used to store scheduler accounting information, and is most likely to be updated in the activations, being invoked by some other `SCont` or the RTS. This is the reason why we model aux-state as a `TVar` and allow it to be modified by some other `SCont`. If the target of the `setAux` is running in another HEC, no rule applies, and we raise a runtime exception. This is reasonable: one HEC should not be poking into another running HEC's state. The rules for `getAux` also have two cases.

An `SCont`'s activations can be invoked using the dequeueAct and enqueueAct primitives. Invoking an `SCont`'s own activation is straight-forward; the activation is fetched from the local state and applied to the current `SCont` (rules INVOKEDEQUEUEACTSELF and INVOKEDEQUEUEACTOTHER). We do allow activations of an `SCont` other than the current `SCont` to be invoked (rule INVOKEDEQUEUEACTOTHER and INOKEENQUEUEACTOTHER). Notice that in order to invoke the activations of other `SConts`, the `SCont` must be passive on the heap, and currently not running.

We allow an `SCont` to modify its *own* activations, and potentially migrate to another ULS. In addition, updating own activations allows initial thread evaluating the `main` IO computation to initialise its activations, and participate in user-level scheduling. In the common use case, once an `SCont`'s activations are initialised, we don't expect it to change. Hence, we do not store the activations in a `TVar`, but rather directly in the underlying TSO object field. The avoids the overheads of transactional access of activations.